1584. Min Cost to Connect All Points

ategory: Advanced Graphs | LeetCode: https://leetcode.com/problems/min-cost-

Problem

- Connect all 2D points with minimum total Manhattan distance $|x1-x2| + |y1-y2|$. Return that cost (a minimum spanning tree).

Intuition

- The graph is complete (every pair is an edge), so dense Prim's algorithm in O(n^2) without a heap is ideal.
- Keep dist[v] = cheapest edge from the tree to v; repeatedly add the closest outside point.

Brute Force

- Kruskal: sort all n^2/2 edges, O(n^2 log n) - fine but slower and memory-heavy.

Optimal Approach

- Dist[] = infinity, dist[0] = 0, inTree[] = false.
- Repeat n times: pick the unvisited u with smallest dist; add it (cost += dist[u]).
- For every unvisited v: dist[v] = min(dist[v], manhattan(u, v)).

Worked Example

```
[[0,0],[2,2],[3,10],[5,2],[7,0]].

add (0,0) cost 0
closest: (2,2) d=4      cost 4
closest: (5,2) d=3      cost 7
closest: (7,0) d=4      cost 11
closest: (3,10) d=9     cost 20
```

Java Solution

```java
public class MinCostToConnectAllPoints {
    public int minCostConnectPoints(int[][] pts) {
        int n = pts.length, cost = 0;
        int[] dist = new int[n];
        boolean[] in = new boolean[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[0] = 0;
        for (int iter = 0; iter < n; iter++) {
            int u = -1;
            for (int v = 0; v < n; v++) if (!in[v] && (u == -1 || dist[v] < dist[u])) u = v;
            in[u] = true;
            cost += dist[u];
            for (int v = 0; v < n; v++) {
                if (in[v]) continue;
                int d = Math.abs(pts[u][0] - pts[v][0]) + Math.abs(pts[u][1] - pts[v][1]);
                if (d < dist[v]) dist[v] = d;
            }
        }
        return cost;
    }
}
```

Complexity

- Time: O(n^2). Space: O(n).

Pitfalls

- A heap-based Prim with all edges is O(n^2 log n) on a complete graph - the array version is faster here.